

**LỰA CHỌN VÀ THỰC HIỆN
CHUYÊN ĐỀ THUẬT TOÁN NÂNG CAO**

Tài liệu dùng để hướng dẫn học viên đăng ký, triển khai, kiểm thử, báo cáo và bảo vệ chuyên đề

Phạm vi	05 nhóm nội dung, 11 chuyên đề
Đối tượng	Học viên cao học – Các thuật toán tối ưu
Hình thức	Làm việc theo nhóm, có mã nguồn, kiểm thử
Ngày hoàn thiện	17/08/2026

MỤC LỤC

- I. Mục tiêu, phạm vi và chuẩn đầu ra chung
- II. Quy định lựa chọn và tổ chức nhóm
- III. Quy trình thực hiện và sản phẩm phải nộp
- IV. Cấu trúc báo cáo, trình bày và đánh giá
- V. Danh mục 11 chuyên đề
- VI. checklist nghiệm thu

I. MỤC TIÊU, PHẠM VI VÀ CHUẨN ĐẦU RA CHUNG

Mục tiêu chung: Học viên không chỉ cài đặt được thuật toán mà còn giải thích được điều kiện áp dụng, chứng minh tính đúng, đánh giá độ phức tạp, xây dựng kiểm thử và liên hệ với một bài toán ứng dụng có mô hình phù hợp.

Sau khi hoàn thành chuyên đề, mỗi nhóm phải chứng minh được các năng lực sau:

- Phát biểu chính xác bài toán, dữ liệu vào, dữ liệu ra, ràng buộc và giả thiết.
- Trình bày ý tưởng thuật toán bằng bất biến, công thức truy hồi hoặc cấu trúc dữ liệu phù hợp.
- Giải thích hoặc chứng minh tính đúng; không chỉ mô tả mã nguồn.
- Phân tích độ phức tạp thời gian và không gian, chỉ rõ biến nào quyết định kích thước bài toán.
- Cài đặt có cấu trúc, có xử lý trường hợp biên, không phụ thuộc vào dữ liệu mẫu duy nhất.
- Thiết kế kiểm thử có hệ thống và có ít nhất một phương pháp đối chiếu độc lập.
- Thực nghiệm có số liệu, nêu môi trường chạy và giải thích kết quả.
- Trình bày được giới hạn, trường hợp không áp dụng và hướng mở rộng.

Ma trận chuyên đề

STT	Chuyên đề	Độ khó	Kiến thức tiên quyết	Quy mô nhóm
1	Chặt nhị phân	Trung bình	Mảng, tham lam, hàm đơn điệu	3
2	Tìm kiếm tam phân	Trung bình	Hàm một biến, số thực	3
3	Sliding Window & Two Pointers	Trung bình	Mảng/chuỗi, Hash Map	3
4	Tree DP & Re-rooting	Khó vừa	Cây, DFS, DP	3–4
5	Bitmask DP	Khó vừa	DP, phép toán bit	3–4
6	Segment Tree & Lazy	Khó vừa	Đệ quy, cây, truy vấn đoạn	3–4
7	Fenwick Tree	Trung bình	Tổng tiền tố, bit	3
8	Tarjan/Low-link	Khó	DFS đồ thị, stack	4–5
9	Max Flow / Min Cut	Khó	Đồ thị, BFS/DFS	3–4
10	KMP và Trie	Trung bình–khó	Chuỗi, cây	3–4
11	Frequent Itemset Mining	Khó	Tập hợp, đếm, dữ liệu giao dịch	4–5

II. QUY ĐỊNH LỰA CHỌN VÀ TỔ CHỨC NHÓM

- Mỗi nhóm gồm 3–5 học viên. Những chuyên đề rất khó nên ưu tiên nhóm 4–5 học viên; nhóm ít người phải thu hẹp phần mở rộng nhưng không được bỏ phần bắt buộc.
- Mỗi nhóm đăng ký một chuyên đề
- Mỗi chuyên đề chỉ giao cho một nhóm.
- Việc phân công công việc trong nhóm phải được ghi trong báo cáo. Tất cả thành viên đều phải hiểu mô hình, thuật toán, mã nguồn và tham gia trả lời khi được hỏi
- Không chọn đề tài chỉ dựa trên “mã nguồn có sẵn”. Nhóm phải chứng minh được khả năng kiểm thử, sửa lỗi và giải thích cài đặt của chính mình.

Vai trò tối thiểu trong nhóm

- **Trưởng nhóm:** lập kế hoạch, tích hợp sản phẩm, quản lý phiên bản và tiến độ.
- **Phụ trách lý thuyết:** chuẩn hóa định nghĩa, chứng minh tính đúng và độ phức tạp.
- **Phụ trách cài đặt:** thiết kế mô-đun, chuẩn hóa giao diện, xử lý lỗi và tối ưu.
- **Phụ trách kiểm thử – thực nghiệm:** xây dựng test, đối chiếu brute force/baseline, thu thập số liệu.
- **Phụ trách báo cáo – trình bày:** biên tập tài liệu, slide và kịch bản demo; vai trò này có thể kiêm nhiệm.

III. QUY TRÌNH THỰC HIỆN VÀ SẢN PHẨM PHẢI NỘP

Bước 1 – Đăng ký

Đăng kí chuyên đề.

Bước 2 – Đề cương kỹ thuật

Mô tả bài toán, giả thiết, thuật toán dự kiến, test đối chiếu và phân mở rộng; dung lượng 2–3 trang.

Bước 3 – Cài đặt phiên bản đúng

Hoàn thành phiên bản cơ sở, có test biên và đối chiếu với brute force hoặc lời giải độc lập trên dữ liệu nhỏ.

Bước 4 – Thực nghiệm

Xây dựng bộ dữ liệu có nhiều quy mô, đo thời gian/bộ nhớ hoặc số thao tác đặc trưng; không chỉ chụp một lần chạy.

Bước 5 – Báo cáo và demo

Nộp báo cáo, mã nguồn, dữ liệu, slide; trình bày thuật toán và chạy demo trực tiếp.

Bước 6 – Hỏi – đáp nếu có

Giảng viên có thể chỉ định bất kỳ thành viên nào giải thích một phần mã nguồn, chứng minh hoặc test case.

Sản phẩm bắt buộc

- Báo cáo kỹ thuật 20–30 trang, không tính phụ lục mã nguồn và dữ liệu.
- Mã nguồn có cấu trúc thư mục rõ ràng; khuyến nghị C++17. Ngôn ngữ khác phải bảo đảm thực nghiệm phù hợp với ràng buộc bài toán.
- Tập README mô tả cách biên dịch/chạy, định dạng dữ liệu vào–ra và các tham số.
- Bộ test gồm: test cơ bản, test biên, test đối kháng và test ngẫu nhiên đối chiếu.
- Tập kết quả thực nghiệm dạng bảng; có thông tin môi trường chạy.
- Slide 12–18 trang và bản demo từ 10–15 phút (nếu có).
- Nhật ký sử dụng AI/công cụ hỗ trợ: mục đích, câu hỏi chính, phần đã kiểm chứng và phần do nhóm tự sửa.

Quy tắc sử dụng AI và mã nguồn tham khảo

Được phép dùng AI để hỗ trợ, nhưng không thay thế năng lực của nhóm

Có thể dùng AI để giải thích khái niệm, gợi ý test, phát hiện lỗi, tạo khung mã hoặc hỗ trợ viết tài liệu.

Mọi đoạn mã lấy từ AI hoặc nguồn ngoài phải được đọc, kiểm thử, điều chỉnh và ghi nguồn. Nhóm chịu trách nhiệm hoàn toàn về tính đúng.

Không chấp nhận báo cáo chứa phát biểu mà nhóm không giải thích được, mã nguồn không thể sửa tại chỗ, hoặc số liệu thực nghiệm không tái lập được.

IV. CẤU TRÚC BÁO CÁO, TRÌNH BÀY VÀ ĐÁNH GIÁ

- **1. Giới thiệu và phát biểu bài toán:** Bối cảnh, dữ liệu vào–ra, ràng buộc, ví dụ minh họa, mục tiêu của chuyên đề.
- **2. Cơ sở lý thuyết:** Định nghĩa, bất biến, công thức, tính chất cần dùng; phân biệt rõ điều kiện đủ và điều kiện bắt buộc.
- **3. Thuật toán và chứng minh:** Mã giả, giải thích từng bước, chứng minh tính đúng hoặc lập luận bất biến.
- **4. Phân tích độ phức tạp:** Thời gian, không gian, trường hợp tốt/xấu nếu có; so sánh với baseline.
- **5. Thiết kế cài đặt:** Cấu trúc lớp/hàm, kiểu dữ liệu, xử lý tràn số, chỉ số, dữ liệu không hợp lệ.
- **6. Kiểm thử và thực nghiệm:** Bộ test, phương pháp đối chiếu, dữ liệu, môi trường chạy, bảng/biểu đồ, phân tích kết quả.
- **7. Ứng dụng, giới hạn và mở rộng:** Mô hình hóa ứng dụng, điều kiện để áp dụng, giới hạn và hướng cải tiến.
- **8. Kết luận và phân công:** Kết quả đạt được, phần chưa hoàn thành, đóng góp của từng thành viên, tài liệu tham khảo.

Thang điểm đề xuất

Hạng mục	Điểm	Yêu cầu cốt lõi	Lỗi trừ nặng
Lý thuyết và tính đúng	25	Phát biểu đúng, có chứng minh/bất biến	Nhầm điều kiện áp dụng; sao chép nhưng không hiểu
Cài đặt	20	Mô-đun rõ, chạy đúng, xử lý biên	Mã chỉ chạy dữ liệu mẫu; lỗi kiểu dữ liệu
Kiểm thử và thực nghiệm	20	Có đối chiếu, test ngẫu nhiên, số liệu tái lập	Không có baseline; số liệu không giải thích được
Báo cáo kỹ thuật	15	Mạch lạc, ký hiệu nhất quán, nguồn đầy đủ	Phát biểu mơ hồ; hình/bảng không chú thích
Trình bày và demo	10	Đúng thời gian, demo ổn định, trả lời rõ	Chỉ đọc slide; demo không tái lập
Làm việc nhóm và bảo vệ cá nhân	10	Phân công hợp lý, mọi thành viên nắm bài	Một số thành viên không giải thích được phần đã nhận

NHÓM 1. KỸ THUẬT TÌM KIẾM VÀ TỐI ƯU KHÔNG GIAN NGHIỆM

Chuyên đề 1. Tìm kiếm nhị phân mở rộng và kỹ thuật “chặt nhị phân”

Độ khó: Trung bình	Tiên quyết: Mảng, tham lam, phân tích hàm đơn điệu	Sản phẩm lõi: Lý thuyết + mã nguồn + kiểm thử + thực nghiệm
---------------------------	---	--

Mục tiêu: Chuyển một bài toán tối ưu thành bài toán quyết định có tính đơn điệu; cài đặt được mẫu first true/last true trên miền nguyên và tìm kiếm gần đúng trên miền thực.

Lý thuyết trọng tâm: Định nghĩa hàm quyết định $f(x)$; chứng minh tính đơn điệu; bất biến của khoảng $[L,R]$; phân biệt tìm giá trị nhỏ nhất thỏa điều kiện và lớn nhất thỏa điều kiện; xử lý miền số nguyên, số thực, sai số ϵ và số vòng lặp cố định; tránh tràn số khi tính mid.

Yêu cầu cài đặt: Xây dựng tối thiểu ba hàm/template: first_true trên số nguyên, last_true trên số nguyên và binary_search_real. Giao diện phải quy định rõ trường hợp không tồn tại đáp án. Không viết riêng một đoạn mã chỉ dùng cho dữ liệu mẫu.

Bài toán kiểm chứng: Cho mảng số không âm A và số nguyên K , chia A thành không quá K đoạn liên tiếp sao cho tổng lớn nhất của một đoạn là nhỏ nhất. Dùng feasible(M) tham lam để kiểm tra có thể chia với ngưỡng M hay không; cần tìm kiếm từ $\max(A)$ đến $\text{sum}(A)$.

Kiểm thử và thực nghiệm: Đối chiếu với vét cạn/DP trên N nhỏ; kiểm tra $N=1$, $K=1$, $K=N$, phân tử bằng 0, giá trị lớn và mảng có một phần tử chi phối. Đo số lần gọi feasible và thời gian khi N tăng.

Ứng dụng minh họa: Phân bổ các tác vụ có thứ tự cho K máy/nhóm xử lý nhằm tối thiểu hóa tải lớn nhất, với giả thiết không được thay đổi thứ tự tác vụ.

Mở rộng khuyến khích: Chặt nhị phân theo đáp án cho bài toán tốc độ sản xuất, khoảng cách tối thiểu hoặc năng lực hệ thống; trình bày parametric search.

Lưu ý kỹ thuật

Điều kiện các phân tử không âm là bắt buộc đối với phép kiểm tra tham lam nêu trên. Nếu có số âm, tính đơn điệu và chiến lược chia đoạn phải được phân tích lại.

Chuyên đề 2. Tìm kiếm tam phân (Ternary Search)

Độ khó: Trung bình	Tiên quyết: Hàm một biến, số thực, đánh giá cực trị	Sản phẩm lỗi: Lý thuyết + mã nguồn + kiểm thử + thực nghiệm
---------------------------	--	--

Mục tiêu: Nhận biết đúng bài toán có hàm đơn đỉnh (unimodal), cài đặt tìm cực tiểu/cực đại trên miền thực và miền nguyên, đồng thời phân biệt với tìm kiếm nhị phân.

Lý thuyết trọng tâm: Định nghĩa hàm đơn đỉnh; quan hệ với hàm lồi/lõm trong các trường hợp thường gặp; hai điểm m_1 và m_2 ; quy tắc loại bỏ một phần khoảng; mỗi vòng lặp giữ khoảng có độ dài $2/3$ nên độ phức tạp là $O(\log((R-L)/\epsilon))$ trên miền thực hoặc $O(\log N)$ trên miền rời rạc; xử lý vùng bằng phẳng.

Yêu cầu cài đặt: Cài đặt hàm tìm cực tiểu và cực đại trên đoạn $[L, R]$. Với miền nguyên, thu hẹp khoảng đến kích thước nhỏ rồi duyệt trực tiếp các giá trị còn lại. Với miền thực, dùng ϵ hoặc số vòng lặp cố định và trả về cả vị trí lẫn giá trị hàm.

Bài toán kiểm chứng: Hai điểm chuyển động thẳng đều trên mặt phẳng: $P_1(t)=P_1+V_1t$, $P_2(t)=P_2+V_2t$, $t \in [0, T]$. Tìm thời điểm làm bình phương khoảng cách giữa hai điểm nhỏ nhất. So sánh kết quả số với nghiệm giải tích khi vận tốc tương đối khác 0.

Kiểm thử và thực nghiệm: Kiểm tra hai điểm đứng yên, vận tốc tương đối bằng 0, nghiệm nằm ở biên $t=0$ hoặc $t=T$, nghiệm nằm trong đoạn và tọa độ lớn. Đánh giá sai số tuyệt đối của t và giá trị khoảng cách.

Ứng dụng minh họa: Tối ưu một tham số liên tục trong mô phỏng chuyển động hoặc hàm chi phí một biến khi đã xác lập được tính đơn đỉnh.

Mở rộng khuyến khích: So sánh tìm kiếm tam phân với đạo hàm/giải tích; áp dụng cho khoảng cách từ điểm chuyển động đến đoạn thẳng hoặc tối ưu vị trí trên một tuyến.

Lưu ý kỹ thuật

Không được dùng ternary search chỉ vì hàm “trông có vẻ lồi”. Nhóm phải nêu lập luận hoặc giả thiết bảo đảm hàm đơn đỉnh trên đúng miền tìm kiếm.

Chuyên đề 3. Cửa sổ trượt (Sliding Window) và Hai con trỏ (Two Pointers)

Độ khó: Trung bình	Tiên quyết: Mảng/chuỗi, Hash Map hoặc mảng tần suất	Sản phẩm lõi: Lý thuyết + mã nguồn + kiểm thử + thực nghiệm
---------------------------	--	--

Mục tiêu: Thiết kế một đoạn $[L, R]$ thay đổi động, duy trì điều kiện hợp lệ và chứng minh tổng số lần dịch chuyển hai biên là tuyến tính.

Lý thuyết trọng tâm: Bất biến của cửa sổ; điều kiện mở rộng R và thu hẹp L ; phân tích khâu hao: mỗi phần tử được thêm và loại bỏ không quá một số lần hằng; phân biệt cửa sổ có độ dài cố định, biến đổi và hai con trỏ trên mảng đã sắp xếp.

Yêu cầu cài đặt: Cài đặt bằng Hash Map hoặc mảng đếm tần suất; tách rõ hàm add/remove; theo dõi số loại phần tử hiện có; không quét lại toàn bộ cửa sổ ở mỗi bước.

Bài toán kiểm chứng: Tìm chuỗi con liên tiếp dài nhất chứa không quá K ký tự phân biệt; trả về độ dài và vị trí đoạn đạt được.

Kiểm thử và thực nghiệm: Kiểm tra chuỗi rỗng, $K=0$, $K=1$, K lớn hơn số ký tự phân biệt, chuỗi toàn ký tự giống nhau, ký tự lặp theo chu kỳ. Đối chiếu $O(N^2)$ trên dữ liệu nhỏ và đo thời gian trên dữ liệu lớn.

Ứng dụng minh họa: Phân tích log theo cửa sổ thời gian: duy trì các sự kiện có timestamp nằm trong khoảng T giây gần nhất và thống kê theo loại sự kiện.

Mở rộng khuyến khích: Kết hợp deque đơn điệu để tìm max/min cửa sổ; bài toán đoạn ngắn nhất thỏa tổng với dữ liệu không âm.

Lưu ý kỹ thuật

Cần nêu rõ điều kiện dữ liệu. Một số bài toán tổng đoạn chỉ dùng được sliding window khi các phần tử không âm.

NHÓM 2. QUY HOẠCH ĐỘNG

Chuyên đề 4. Quy hoạch động trên cây (Tree DP) và kỹ thuật Re-rooting

Độ khó: Khó	Tiên quyết: Cây, DFS, quy hoạch động	Sản phẩm lõi: Lý thuyết + mã nguồn + kiểm thử + thực nghiệm
--------------------	---	--

Mục tiêu: Tính thông tin từ con lên cha bằng DFS hậu thứ tự và chuyển đáp án giữa hai gốc kề nhau để thu được đáp án cho mọi đỉnh trong $O(N)$.

Lý thuyết trọng tâm: Kích thước cây con $sz[u]$; tổng khoảng cách trong cây con; hai lượt DFS; công thức chuyển gốc $ans[v]=ans[u]+N-2\cdot sz[v]$ khi v là con của u trong cây không trọng số; chứng minh vì các đỉnh trong cây con v gần hơn 1, các đỉnh còn lại xa hơn 1.

Yêu cầu cài đặt: Biểu diễn danh sách kề; cài đặt hai lượt DFS hoặc một khung reroot tổng quát. Phải xử lý N lớn bằng cách cân nhắc giới hạn đệ quy/stack; dùng kiểu số đủ lớn cho tổng khoảng cách.

Bài toán kiểm chứng: Với mỗi đỉnh u của cây N đỉnh, tính tổng khoảng cách từ u đến tất cả các đỉnh khác.

Kiểm thử và thực nghiệm: Kiểm tra cây một đỉnh, đường thẳng, hình sao, cây cân bằng và cây ngẫu nhiên. Đối chiếu bằng BFS từ từng đỉnh trên N nhỏ; đo thời gian $O(N)$ so với $O(N^2)$.

Ứng dụng minh họa: Chọn vị trí trung tâm trên một mạng dạng cây để giảm tổng khoảng cách tới các điểm phục vụ, trong mô hình mà tuyến kết nối không tạo chu trình.

Mở rộng khuyến khích: Cây có trọng số; tính giá trị khác theo gốc; xây dựng khung reroot dùng prefix/suffix để gộp đóng góp từ các nhánh.

Lưu ý kỹ thuật

Ứng dụng chỉ hợp lý khi mạng được mô hình hóa là cây hoặc đã chọn một cây khung phù hợp. Không suy rộng trực tiếp sang đồ thị bất kỳ.

Chuyên đề 5. Quy hoạch động trạng thái (Bitmask DP)

Độ khó: Khó	Tiên quyết: Quy hoạch động, thao tác bit, đồ thị trọng số	Sản phẩm lỗi: Lý thuyết + mã nguồn + kiểm thử + thực nghiệm
--------------------	--	--

Mục tiêu: Biểu diễn một tập con bằng bitmask, xây dựng trạng thái kết hợp tập đã chọn và vị trí cuối, khôi phục phương án tối ưu.

Lý thuyết trọng tâm: Các phép $\ll$, $\gg$, $\&$, $|$, $\wedge$; kiểm tra/đặt/xóa bit; trạng thái $DP[mask][i]$; chuyển từ mask sang $mask \cup \{j\}$; phân biệt độ phức tạp TSP $O(N^2 \cdot 2^N)$ với kỹ thuật duyệt mọi submask của mọi mask $O(3^N)$.

Yêu cầu cài đặt: Cài đặt TSP với INF an toàn, mảng cha để khôi phục chu trình/đường đi. Khuyến nghị $N \leq 18$ cho cấu hình thông thường; chỉ dùng $N \leq 20$ khi đã tính toán bộ nhớ và tối ưu cách lưu.

Bài toán kiểm chứng: Bài toán người chào hàng trên đồ thị đầy đủ hoặc ma trận chi phí; xuất chi phí nhỏ nhất và thứ tự thăm các đỉnh. Phải quy định rõ có quay về điểm đầu hay không.

Kiểm thử và thực nghiệm: Đối chiếu hoán vị trên $N \leq 10$; kiểm tra đồ thị đối xứng/bất đối xứng, cạnh không tồn tại, $N=1$, nhiều phương án cùng tối ưu. Báo cáo bộ nhớ thực tế theo N .

Ứng dụng minh họa: Lập lịch một số lượng nhỏ tác vụ có chi phí chuyển đổi phụ thuộc tác vụ trước đó; tối ưu thứ tự thực hiện.

Mở rộng khuyến khích: Hamiltonian path, Steiner tree trên N nhỏ, DP theo tập con cho phân hoạch; tối ưu bộ nhớ theo lớp mask.

Lưu ý kỹ thuật

Với $N=20$, bảng $2^N \cdot N$ có hơn 20 triệu trạng thái; kiểu dữ liệu và cấu trúc lưu trữ quyết định trực tiếp khả năng chạy.

NHÓM 3. CẤU TRÚC DỮ LIỆU NÂNG CAO
Chuyên đề 6. Cây phân đoạn (Segment Tree) và Lazy Propagation

Độ khó: Khó	Tiên quyết: Độ quy, truy vấn đoạn, cây nhị phân	Sản phẩm lỗi: Lý thuyết + mã nguồn + kiểm thử + thực nghiệm
--------------------	--	--

Mục tiêu: Duy trì thông tin tổng hợp trên các đoạn, hỗ trợ cập nhật đoạn và truy vấn đoạn trong $O(\log N)$ bằng trạng thái lazy.

Lý thuyết trọng tâm: Mỗi nút đại diện một đoạn; phép gộp; độ cao $O(\log N)$; mảng $4N$ là cận an toàn phổ biến; lazy lưu phép cập nhật chưa đẩy xuống con; bất biến giữa giá trị nút và nhãn lazy; điều kiện đoạn nằm ngoài, phủ toàn bộ và giao một phần.

Yêu cầu cài đặt: Đóng gói lớp SegmentTree với build, update_range, query_range, apply và push_down. Tách rõ loại dữ liệu của giá trị cây và giá trị cập nhật; dùng 64-bit cho tổng.

Bài toán kiểm chứng: Hỗ trợ hai thao tác: cộng V vào mọi phần tử trong $[L,R]$ và trả về tổng các phần tử trong $[L,R]$.

Kiểm thử và thực nghiệm: Kiểm tra cập nhật toàn đoạn, một điểm, nhiều đoạn chồng lấn, giá trị âm, N không phải lũy thừa của 2. Đối chiếu với mảng thường trên test ngẫu nhiên; đo thời gian theo số thao tác.

Ứng dụng minh họa: Duy trì tổng hoặc cực trị trên chuỗi dữ liệu thời gian khi có cập nhật theo khoảng và truy vấn tức thời.

Mở rộng khuyến khích: Range assignment, affine update, min/max; segment tree động hoặc persistent segment tree.

Lưu ý kỹ thuật

Không gọi Segment Tree là cây nhị phân hoàn toàn trong mọi trường hợp. Điều quan trọng là phân hoạch đoạn và chiều cao logarit.

Chuyên đề 7. Cây Fenwick (Binary Indexed Tree)

Độ khó: Trung bình	Tiên quyết: Tổng tiền tố, đánh chỉ số 1-based, thao tác bit	Sản phẩm lỗi: Lý thuyết + mã nguồn + kiểm thử + thực nghiệm
---------------------------	--	--

Mục tiêu: Hiểu lowbit và cấu trúc các đoạn tiền tố cục bộ; cài đặt cập nhật điểm và truy vấn tổng tiền tố trong $O(\log N)$.

Lý thuyết trọng tâm: Biểu thức $\text{lowbit}(x) = x \& (-x)$; ý nghĩa đoạn mà $\text{tree}[x]$ quản lý; update đi lên bằng $x += \text{lowbit}(x)$, query đi xuống bằng $x -= \text{lowbit}(x)$; so sánh với Segment Tree về không gian, hằng số và khả năng hỗ trợ phép toán.

Yêu cầu cài đặt: Xây dựng lớp Fenwick với `add`, `prefix_sum`, `range_sum`; xử lý rõ chỉ số 0-based ở giao diện và 1-based bên trong. Không đặt tiêu chí số dòng; ưu tiên tính đúng và khả năng tái sử dụng.

Bài toán kiểm chứng: Đếm số nghịch thế của mảng A. Phải dùng nén tọa độ; quy định nghịch thế là cặp $i < j$ và $A[i] > A[j]$, không đếm phần tử bằng nhau.

Kiểm thử và thực nghiệm: Kiểm tra mảng tăng, giảm, có phần tử trùng, số âm và giá trị rất lớn. Đối chiếu $O(N^2)$ trên N nhỏ; đo thời gian N lớn.

Ứng dụng minh họa: Duy trì tần suất động để tính xếp hạng hoặc tổng số phần tử không vượt quá một giá trị; có thể dùng trong bảng xếp hạng thời gian thực.

Mở rộng khuyến khích: Tìm phần tử thứ k bằng binary lifting trên Fenwick; Fenwick 2D; so sánh thực nghiệm với Segment Tree.

Lưu ý kỹ thuật

Ứng dụng trong mã hóa số học chỉ phù hợp khi Fenwick được dùng để duy trì bảng tần suất tích lũy động; cần mô tả đúng vai trò này.

NHÓM 4. THUẬT TOÁN ĐỒ THỊ VÀ LUỒNG TRÊN MẠNG

Chuyên đề 8. Thuật toán Tarjan/Low-link: điểm khớp, cầu và thành phần liên thông mạnh

Độ khó: Rất khó	Tiên quyết: DFS đồ thị, cạnh có hướng/vô hướng, stack	Sản phẩm lõi: Lý thuyết + mã nguồn + kiểm thử + thực nghiệm
------------------------	--	--

Mục tiêu: Nắm bản chất low-link và cài đặt hai mô-đun độc lập: (A) điểm khớp/cầu trên đồ thị vô hướng; (B) SCC và đồ thị cô đặc trên đồ thị có hướng.

Lý thuyết trọng tâm: Mảng `num/disc` và `low`; cạnh cây và cạnh ngược; điều kiện $\text{low}[v] > \text{num}[u]$ cho cầu; $\text{low}[v] \geq \text{num}[u]$ cho điểm khớp không phải gốc; gốc DFS là điểm khớp khi có ít nhất hai con DFS. Với SCC: `stack`, `onStack`, điều kiện $\text{low}[u] = \text{num}[u]$ để tách một SCC. Đồ thị cô đặc là DAG.

Yêu cầu cài đặt: Dùng `edge id` để xử lý đúng cạnh song song trong đồ thị vô hướng. Viết riêng hàm `find_bridges_articulations` và `find_scc`; mô-đun SCC phải sinh `component_id` và xây `condensation graph` loại bỏ cạnh trùng.

Bài toán kiểm chứng: Phần A: liệt kê điểm khớp và cầu của đồ thị vô hướng. Phần B: tìm SCC của đồ thị có hướng và xây đồ thị cô đặc.

Kiểm thử và thực nghiệm: Kiểm tra đồ thị rỗng cạnh, đường thẳng, chu trình, nhiều thành phần, cạnh song song, self-loop và đồ thị có hướng với SCC lồng ghép. Đối chiếu cầu bằng cách xóa từng cạnh trên N nhỏ; đối chiếu SCC bằng reachability hai chiều.

Ứng dụng minh họa: Phát hiện điểm đơn lỗi trong mạng vô hướng; gom các mô-đun phụ thuộc vòng thành SCC để tạo DAG phụ thuộc.

Mở rộng khuyến khích: Thành phần song liên thông, bridge tree, 2-SAT dựa trên SCC.

Lưu ý kỹ thuật

Trong DFS của đồ thị vô hướng, không nên áp dụng máy móc phân loại forward/cross edge như đồ thị có hướng. Phải xử lý cạnh cha bằng edge id, đặc biệt khi có cạnh song song.

Chuyên đề 9. Luồng cực đại trên mạng (Max Flow / Min Cut)

Độ khó: Khó	Tiên quyết: Đồ thị có hướng, BFS/DFS, kiểu số nguyên lớn	Sản phẩm lõi: Lý thuyết + mã nguồn + kiểm thử + thực nghiệm
--------------------	---	--

Mục tiêu: Mô hình hóa năng lực, xây residual graph và tìm luồng cực đại; trích xuất lát cắt nhỏ nhất sau khi thuật toán kết thúc.

Lý thuyết trọng tâm: Cạnh thuận/cạnh nghịch, capacity dư, augmenting path; bảo toàn luồng; định lý Max Flow–Min Cut; so sánh Ford–Fulkerson dùng DFS với Edmonds–Karp dùng BFS và độ phức tạp $O(VE^2)$ cho năng lực nguyên.

Yêu cầu cài đặt: Xây danh sách kề với cấu trúc cạnh chứa to, rev và residual capacity. Cài đặt Edmonds–Karp bắt buộc; Dinic là phần mở rộng. Phải dùng kiểu số phù hợp và tránh sửa sai cặp cạnh nghịch.

Bài toán kiểm chứng: Mô hình hóa cặp ghép cực đại hai phía bằng nguồn–trái–phải–đích. Ngoài ra, phải kiểm thử ít nhất một mạng tổng quát có năng lực lớn hơn 1 và trích xuất min-cut.

Kiểm thử và thực nghiệm: Kiểm tra mạng không có đường s–t, nhiều đường tăng, cạnh song song, năng lực 0, bottleneck và năng lực lớn. Đối chiếu matching bằng thuật toán Kuhn trên đồ thị nhỏ.

Ứng dụng minh họa: Phân công người–việc, phân phối năng lực hoặc định tuyến trong mô hình tĩnh với giới hạn dung lượng.

Mở rộng khuyến khích: Dinic, circulation with demands, min-cost max-flow.

Lưu ý kỹ thuật

Ford–Fulkerson dùng DFS có thể phụ thuộc cách chọn đường tăng; với năng lực không nguyên còn có vấn đề hội tụ. Phạm vi bắt buộc nên dùng năng lực nguyên và Edmonds–Karp.

NHÓM 5. XỬ LÝ CHUỖI VÀ KHAI PHÁ DỮ LIỆU

Chuyên đề 10. Xử lý chuỗi với KMP và cấu trúc Trie

Độ khó: Trung bình–khó	Tiên quyết: Chuỗi, mảng, cây, quản lý bộ nhớ	Sản phẩm lõi: Lý thuyết + mã nguồn + kiểm thử + thực nghiệm
-------------------------------	---	--

Mục tiêu: Cài đặt hai kỹ thuật cho hai lớp bài toán khác nhau: KMP cho so khớp mẫu tuyến tính và Trie cho lưu/truy vấn theo tiền tố.

Lý thuyết trọng tâm: KMP: hàm tiền tố π /LPS, ý nghĩa độ dài biên dài nhất, cách tái sử dụng kết quả khi mismatch và độ phức tạp $O(n+m)$. Trie: mỗi nút biểu diễn một tiền tố, thao tác insert/search/starts_with; độ phức tạp theo độ dài chuỗi và phụ thuộc kích thước bảng chữ cái/cách lưu cạnh.

Yêu cầu cài đặt: Mô-đun KMP phải sinh LPS và trả về tất cả vị trí xuất hiện, kể cả các lần xuất hiện chồng lấn. Mô-đun Trie phải hỗ trợ insert, exact search, prefix query và autocomplete giới hạn số kết quả; quy định rõ bảng chữ cái.

Bài toán kiểm chứng: Bài A: tìm mọi vị trí của mẫu trong văn bản bằng KMP. Bài B: xây từ điển hỗ trợ autocomplete các từ có cùng tiền tố. Hai bài đều bắt buộc.

Kiểm thử và thực nghiệm: KMP: mẫu dài hơn văn bản, mẫu lặp, xuất hiện chồng lấn, không xuất hiện. Trie: từ trùng, một từ là tiền tố của từ khác, tiền tố rỗng và ký tự ngoài bảng chữ cái. So sánh với hàm tìm kiếm chuẩn/duyet danh sách.

Ứng dụng minh họa: Tìm mẫu trong log/văn bản; tra cứu tiền tố, autocomplete hoặc lọc từ khóa trong một từ điển đã chuẩn hóa.

Mở rộng khuyến khích: Aho–Corasick cho nhiều mẫu; compressed trie/radix tree; hỗ trợ Unicode bằng ánh xạ ký tự.

Lưu ý kỹ thuật

KMP và Trie không phải một thuật toán thống nhất. Báo cáo phải tách mục tiêu, bất biến, độ phức tạp và bài kiểm chứng của từng mô-đun.

Chuyên đề 11. Khai phá tập phổ biến (Frequent Itemset Mining) với Apriori

Độ khó: Khó	Tiên quyết: Tập hợp, đếm tần suất, dữ liệu giao dịch	Sản phẩm lõi: Lý thuyết + mã nguồn + kiểm thử + thực nghiệm
--------------------	---	--

Mục tiêu: Cài đặt Apriori, phân biệt khai phá tập phổ biến với sinh luật kết hợp, và đánh giá ảnh hưởng của ngưỡng hỗ trợ đến số ứng viên và thời gian chạy.

Lý thuyết trọng tâm: Cơ sở dữ liệu giao dịch; support count và support ratio; tập k -phần tử phổ biến L_k ; tính chống đơn điệu của support: mọi tập con của một tập phổ biến đều phổ biến; phép join và prune để sinh C_k từ L_{k-1} . Confidence chỉ dùng khi sinh luật $X \rightarrow Y$ từ các tập phổ biến đã tìm được.

Yêu cầu cài đặt: Cài đặt sinh C_k từ L_{k-1} , loại ứng viên có tập con $(k-1)$ -phần tử không phổ biến, quét cơ sở dữ liệu để đếm support và xuất toàn bộ tập phổ biến. Có thể dùng biểu diễn tập đã sắp xếp hoặc bitset nhưng phải giải thích.

Bài toán kiểm chứng: Khai phá các tập mặt hàng phổ biến trên một cơ sở dữ liệu giao dịch nhỏ; sau đó sinh một số luật kết hợp thỏa minconf. Báo cáo số lượng C_k , L_k theo từng mức k .

Kiểm thử và thực nghiệm: Đối chiếu vết cạn trên số lượng mặt hàng nhỏ; kiểm tra giao dịch trùng, giao dịch rỗng, ngưỡng minsup tuyệt đối/tương đối, dữ liệu thừa/dày. Thực nghiệm nhiều mức minsup và phân tích bùng nổ ứng viên.

Ứng dụng minh họa: Phân tích giỏ hàng, nhóm sự kiện thường cùng xuất hiện hoặc mẫu đồng xuất hiện trong dữ liệu giao dịch.

Mở rộng khuyến khích: So sánh Apriori với FP-Growth hoặc Eclat; nghiên cứu tập có trọng số hoặc High-Occupancy Itemsets. Nếu làm HOI, phải trình bày rõ occupancy không có tính chống đơn điệu như support và cần cận trên để tỉa nhánh.

Lưu ý kỹ thuật

Không dùng confidence để quyết định một itemset có phổ biến hay không. Cần ghi rõ minsup là số đếm hay tỷ lệ và dùng nhất quán trong toàn bộ chương trình.

Checklist trước khi nộp

- ☐ Bài toán, đầu vào, đầu ra, ràng buộc và giả thiết đã được phát biểu đầy đủ.
- ☐ Mỗi công thức/ký hiệu đều được định nghĩa trước khi sử dụng.
- ☐ Có mã giả và giải thích vì sao thuật toán đúng.
- ☐ Độ phức tạp thời gian/không gian khớp với mã nguồn thực tế.
- ☐ Có test biên và test ngẫu nhiên đối chiếu với lời giải đơn giản.
- ☐ Kết quả thực nghiệm có môi trường chạy, kích thước dữ liệu và đơn vị đo.
- ☐ Ứng dụng được mô hình hóa hợp lý, không phóng đại khả năng của thuật toán.
- ☐ Mã nguồn chạy lại được theo README; không phụ thuộc đường dẫn cá nhân.
- ☐ Tài liệu tham khảo và phần sử dụng AI/công cụ hỗ trợ đã được khai báo.
- ☐ Mọi thành viên có thể giải thích phần được phân công và phần lỗi của thuật toán.

3. Cấu trúc thư mục mã nguồn gợi ý

```
project/
├── README.md
├── src/      # mã nguồn chính
├── tests/    # test cố định và bộ sinh test
├── data/     # dữ liệu đầu vào dùng trong thực nghiệm
├── results/  # bảng kết quả, log chạy
├── report/   # báo cáo và hình minh họa
└── slides/  # tệp trình bày
```

— HẾT —